

Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An autonomous Institution affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	V
Subject Code & Name	ICS1512 - Machine Learning Algorithms Laboratory		
Academic year	2025-2026 (Odd)	Batch: 2023-2028	Due date: 12/10/2025

Experiment 6: Spam Classification with Dimensionality Reduction

1 Aim

To build and evaluate various classification models for spam detection. This experiment investigates the impact of Principal Component Analysis (PCA) on model performance and compares a wide range of classifiers to identify the most effective model for the task.

2 Data Preprocessing

2.1 Data Loading and Cleaning

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.model_selection import train_test_split
5
6 # Load data
7 df = pd.read_csv('spambase_csv.csv')
8 print("Initial DataFrame head:")
9 print(df.head())
10 print("\nInitial shape:", df.shape)
11
12 # Impute missing values with the median
13 df.fillna(df.median(numeric_only=True), inplace=True)
14
15 # Remove outliers using the Z-score method (threshold=3)
16 def remove_outliers(df, threshold=3):
17     numeric_cols = df.select_dtypes(include=[np.number]).columns
18     z_scores = np.abs((df[numeric_cols] - df[numeric_cols].mean()) / df
19                       [numeric_cols].std())
20     return df[(z_scores < threshold).all(axis=1)]
```

```

21 df_cleaned = remove_outliers(df)
22 print("\nShape after cleaning:", df_cleaned.shape)
23
24 # Feature Scaling and Data Splitting
25 X = df_cleaned.drop(columns=['class'])
26 y = df_cleaned['class']
27 scaler = StandardScaler()
28 X_scaled = scaler.fit_transform(X)
29 X_train, X_test, y_train, y_test = train_test_split(
30     X_scaled, y, test_size=0.2, stratify=y, random_state=42
31 )

```

Listing 1: Loading, cleaning, and preprocessing data

```

[commandchars=
{}] word_freq_make word_freq_address word_freq_all word_freq_3d \ 0 0.00 0.64 0.64 0.0 1
0.21 0.28 0.50 0.0 2 0.06 0.00 0.71 0.0 3 0.00 0.00 0.00 0.0 4 0.00 0.00 0.00 0.0
word_freq_our word_freq_over word_freq_remove word_freq_internet \ 0 0.32 0.00 0.00
0.00 1 0.14 0.28 0.21 0.07 2 1.23 0.19 0.19 0.12 3 0.63 0.00 0.31 0.63 4 0.63 0.00 0.31 0.63
word_freq_order word_freq_mail ... char_freq_%3B char_freq_%28 \ 0 0.00 0.00 ...
0.00 0.000 1 0.00 0.94 ... 0.00 0.132 2 0.64 0.25 ... 0.01 0.143 3 0.31 0.63 ... 0.00 0.137
4 0.31 0.63 ... 0.00 0.135
char_freq_%5B char_freq_%21 char_freq_%24 char_freq_%23 \ 0 0.0 0.778 0.000 0.000
1 0.0 0.372 0.180 0.048 2 0.0 0.276 0.184 0.010 3 0.0 0.137 0.000 0.000 4 0.0 0.135 0.000
0.000
capital_run_length_average capital_run_length_longest \ 0 3.756 61 1 5.114 101 2 9.821
485 3 3.537 40 4 3.537 40
capital_run_length_total class 0 278 1 1 1028 1 2 2259 1 3 191 1 4 191 1
[5 rows x 58 columns] Initial shape: (4601, 58)

```

3 Model Training and Evaluation

3.1 Logistic Regression

```

1 from sklearn.linear_model import LogisticRegression
2 from sklearn.metrics import accuracy_score
3
4 lr_model = LogisticRegression(random_state=42)
5 lr_model.fit(X_train, y_train)
6 test_acc = accuracy_score(y_test, lr_model.predict(X_test))
7 print(f"Logistic Regression Test Accuracy: {test_acc:.4f}")

```

Listing 2: Training a Logistic Regression Classifier

Logistic Regression Test Accuracy: 0.9272

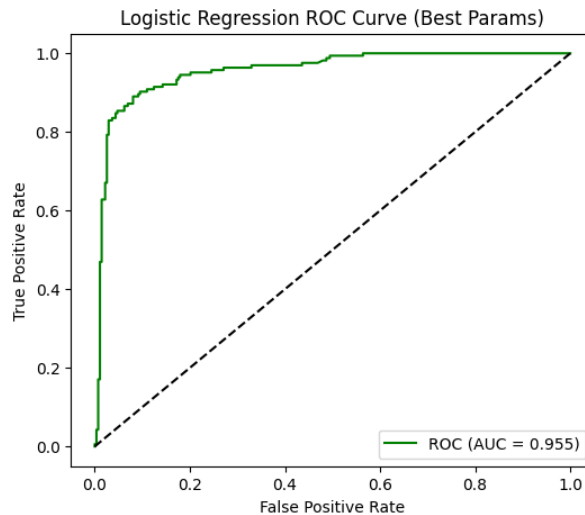


Figure 1: ROC Curve for Logistic Regression model.

3.2 K-Nearest Neighbors (KNN) Classifier

```
1 from sklearn.neighbors import KNeighborsClassifier
2
3 knn_model = KNeighborsClassifier()
4 knn_model.fit(X_train, y_train)
5 test_acc = accuracy_score(y_test, knn_model.predict(X_test))
6 print(f"KNN Test Accuracy: {test_acc:.4f}")
```

Listing 3: Training a KNN Classifier

KNN Performance Table

	k	weights	metric	Accuracy_no_PCA	Accuracy_PCA
0	3	uniform	euclidean	0.899314	0.901602
1	3	uniform	manhattan	0.899314	0.897025
2	3	distance	euclidean	0.910755	0.908467
3	3	distance	manhattan	0.903890	0.906178
4	5	uniform	euclidean	0.892449	0.899314
5	5	uniform	manhattan	0.897025	0.894737
6	5	distance	euclidean	0.908467	0.910755
7	5	distance	manhattan	0.906178	0.906178

Best KNN Params]

```
k          3
weights    distance
metric     euclidean
Accuracy_no_PCA  0.910755
Accuracy_PCA    0.908467
Name: 2, dtype: object
```

KNN Test Accuracy: 0.9108

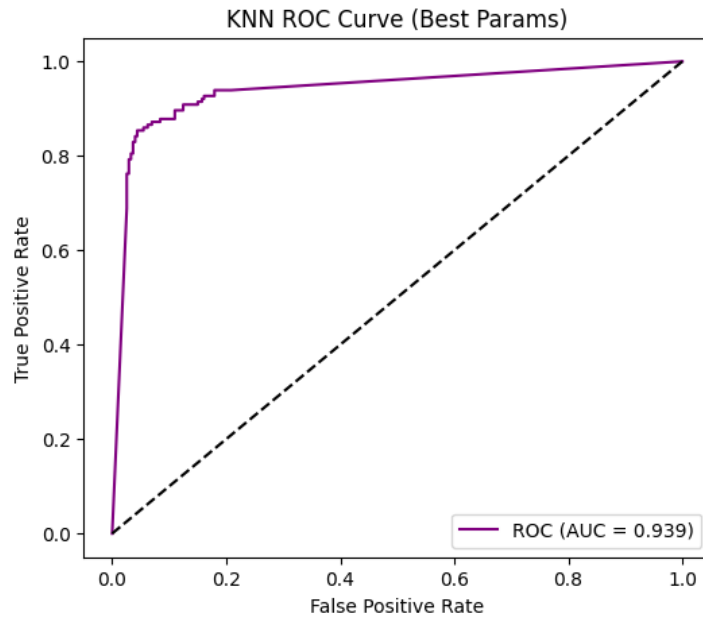


Figure 2: ROC Curve for KNN model.

3.3 Naive Bayes Classifier

```
1 from sklearn.naive_bayes import GaussianNB
2
3 nb_model = GaussianNB()
4 nb_model.fit(X_train, y_train)
5 test_acc = accuracy_score(y_test, nb_model.predict(X_test))
6 print(f"Naive Bayes Test Accuracy: {test_acc:.4f}")
```

Listing 4: Training a Gaussian Naive Bayes Classifier

Naive Bayes Test Accuracy: 0.8353

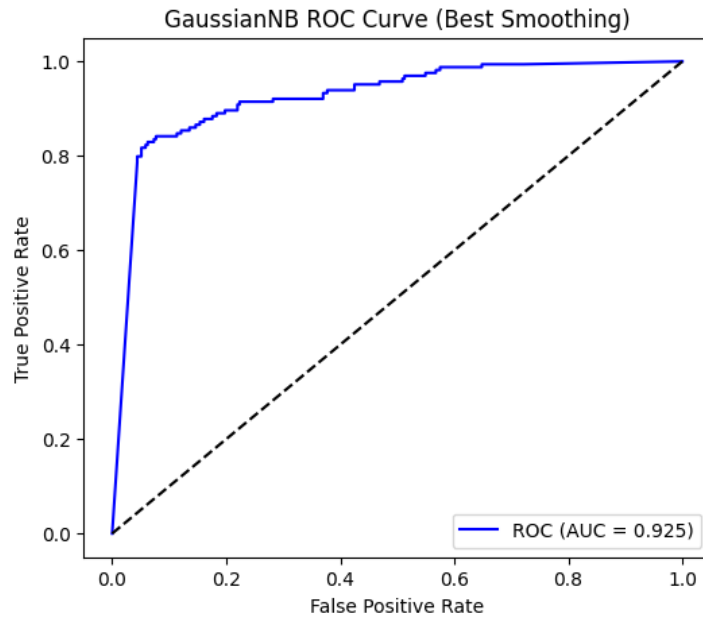


Figure 3: ROC Curve for Naive Bayes model.

3.4 Decision Tree Classifier

```

1 from sklearn.tree import DecisionTreeClassifier
2
3 dt_model = DecisionTreeClassifier(random_state=42)
4 dt_model.fit(X_train, y_train)
5 test_acc = accuracy_score(y_test, dt_model.predict(X_test))
6 print(f"Decision Tree Test Accuracy: {test_acc:.4f}")

```

Listing 5: Training a Decision Tree Classifier

Decision Tree Test Accuracy: 0.9169

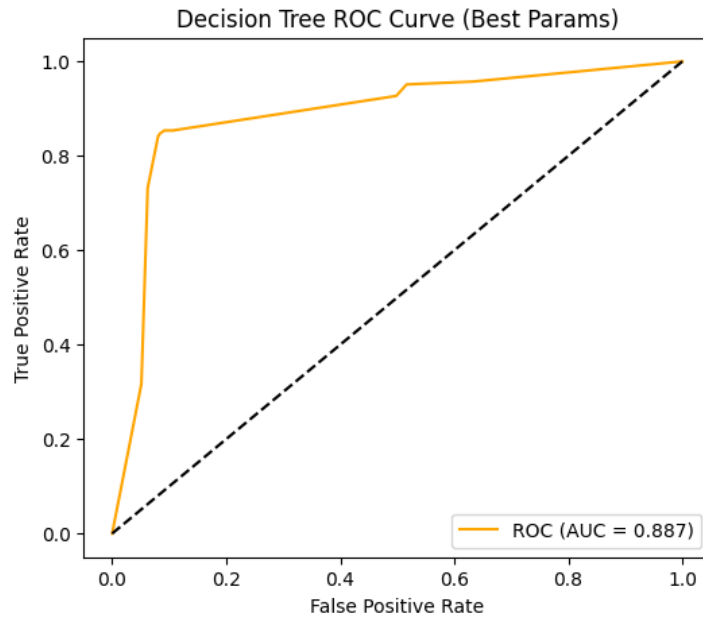


Figure 4: ROC Curve for Decision Tree model.

3.5 Support Vector Machine (SVM)

```
1 from sklearn.svm import SVC
2
3 svm_model = SVC(random_state=42)
4 svm_model.fit(X_train, y_train)
5 test_acc = accuracy_score(y_test, svm_model.predict(X_test))
6 print(f"SVM Test Accuracy: {test_acc:.4f}")
```

Listing 6: Evaluating the SVM Classifier

SVC Performance Table

	kernel	C	gamma	Accuracy	PCA	Accuracy	PCA
0	linear	0.1	scale	0.93135		0.924485	
1	linear	10.0	scale	0.93135		0.924485	
2	rbf	0.1	scale	0.93135		0.924485	
3	rbf	10.0	scale	0.93135		0.924485	

Best Params (No PCA)

```
kernel          linear
C                0.1
gamma           scale
Accuracy PCA    0.93135
Accuracy PCA    0.924485
Name: 0, dtype: object
```

SVM Test Accuracy: 0.9310

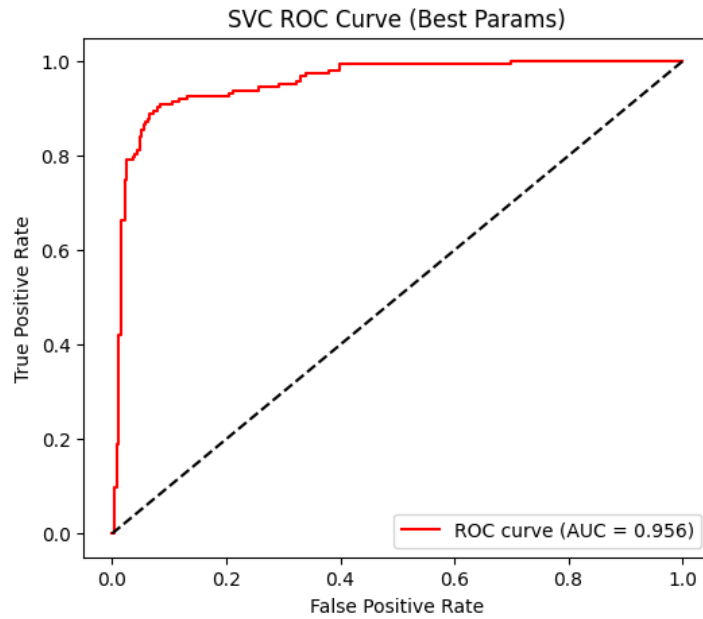


Figure 5: ROC Curve for SVM model.

3.6 Random Forest Classifier

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 rf_model = RandomForestClassifier(random_state=42)
4 rf_model.fit(X_train, y_train)
5 test_acc = accuracy_score(y_test, rf_model.predict(X_test))
6 print(f"Random Forest Test Accuracy: {test_acc:.4f}")
```

Listing 7: Training a Random Forest Classifier

Random Forest Test Accuracy: 0.9246

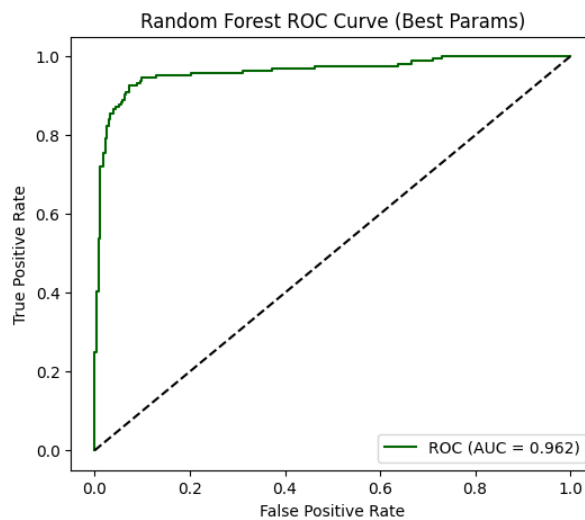


Figure 6: ROC Curve for Random Forest model.

3.7 Gradient Boosting Classifier

```
1 from sklearn.ensemble import GradientBoostingClassifier
2
3 gb_model = GradientBoostingClassifier(random_state=42)
4 gb_model.fit(X_train, y_train)
5 test_acc = accuracy_score(y_test, gb_model.predict(X_test))
6 print(f"Gradient Boosting Test Accuracy: {test_acc:.4f}")
```

Listing 8: Training a Gradient Boosting Classifier

Gradient Boosting Test Accuracy: 0.9310

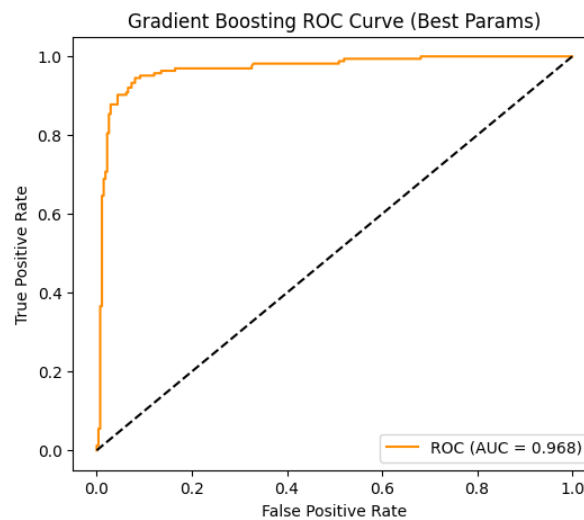


Figure 7: ROC Curve for Gradient Boost model.

3.8 AdaBoost Classifier

```
1 from sklearn.ensemble import AdaBoostClassifier
2
3 ada_model = AdaBoostClassifier(random_state=42)
4 ada_model.fit(X_train, y_train)
5 test_acc = accuracy_score(y_test, ada_model.predict(X_test))
6 print(f"AdaBoost Test Accuracy: {test_acc:.4f}")
```

Listing 9: Training an AdaBoost Classifier

AdaBoost Test Accuracy: 0.9156

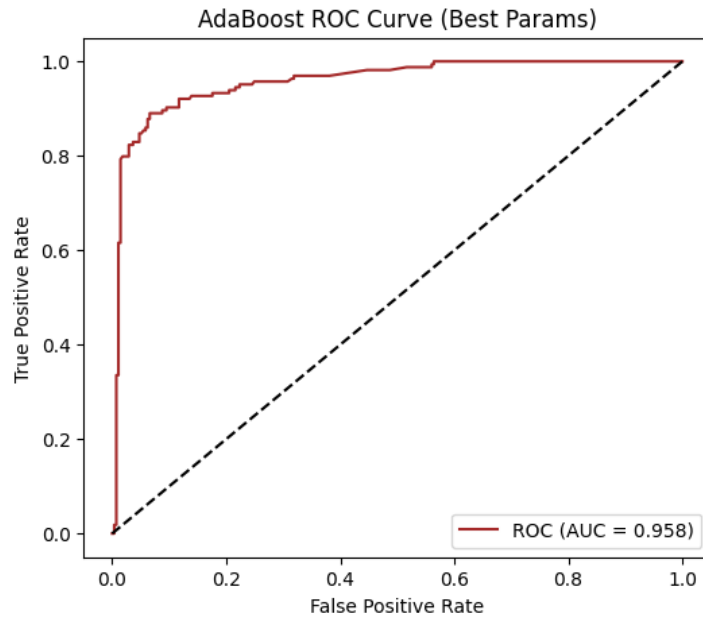


Figure 8: ROC Curve for AdaBoost model.

3.9 XGBoost Classifier

```

1 from xgboost import XGBClassifier
2
3 xgb_model = XGBClassifier(random_state=42, use_label_encoder=False,
4   eval_metric='logloss')
5 xgb_model.fit(X_train, y_train)
6 test_acc = accuracy_score(y_test, xgb_model.predict(X_test))
7 print(f"XGBoost Test Accuracy: {test_acc:.4f}")

```

Listing 10: Training an XGBoost Classifier

XGBoost Test Accuracy: 0.9400

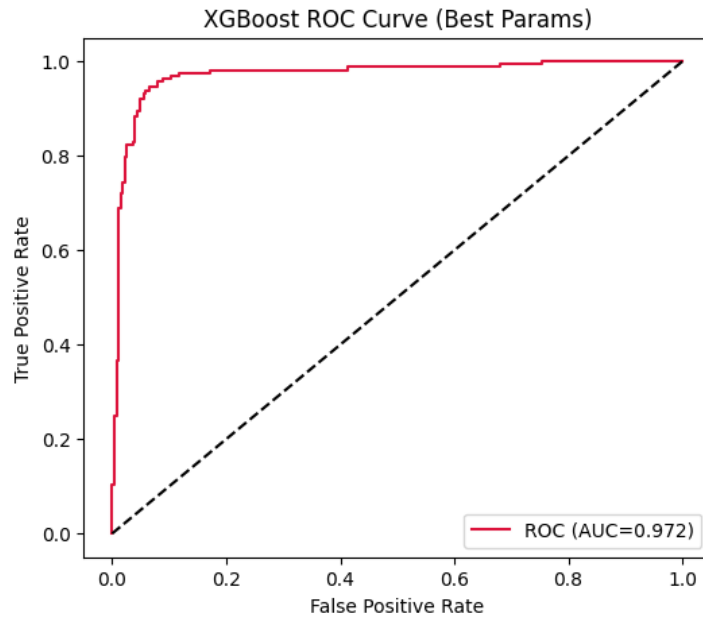


Figure 9: ROC Curve for XGBoost model.

3.10 Stacked Classifier

```

1 from sklearn.ensemble import StackingClassifier
2
3 base_estimators = [
4     ('rf', RandomForestClassifier(random_state=42)),
5     ('gb', GradientBoostingClassifier(random_state=42))
6 ]
7 stacked_model = StackingClassifier(estimators=base_estimators,
8     final_estimator=LogisticRegression())
9 stacked_model.fit(X_train, y_train)
10 test_acc = accuracy_score(y_test, stacked_model.predict(X_test))
11 print(f"Stacked Classifier Test Accuracy: {test_acc:.4f}")

```

Listing 11: Training a Stacked Classifier

Stacked Classifier Test Accuracy: 0.9156

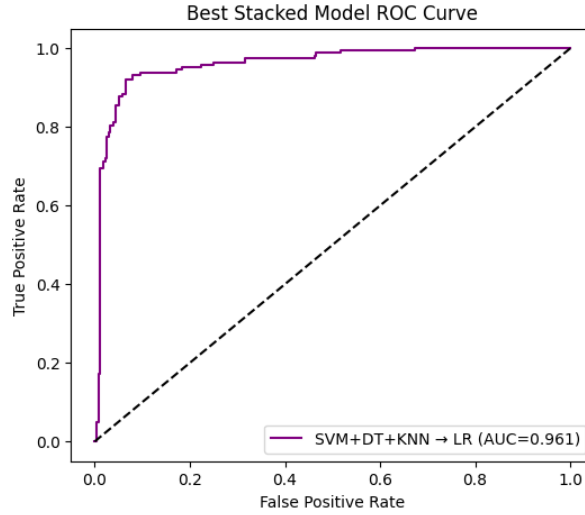


Figure 10: ROC Curve for Stacked model.

4 Model Comparison

The performance of all models was compared to identify the most effective classifier. The results clearly show the superiority of ensemble methods, especially gradient-boosted trees.

5 Cross-Validation and Final Model Comparison

5.1 5-Fold Cross-Validation Results

5-Fold Cross-Validation Results

	Model	Fold1	Fold2	Fold3	Fold4	Fold5	Avg (No PCA)	Avg (Wi
8	XGBoost	0.9405	0.9268	0.9497	0.9519	0.9634	0.9465	
5	Random Forest	0.9291	0.9108	0.9451	0.9405	0.9519	0.9355	
7	Gradient Boosting	0.9199	0.9176	0.9451	0.9359	0.9519	0.9341	
9	Stacked	0.9176	0.9108	0.9382	0.9336	0.9359	0.9272	
6	AdaBoost	0.9085	0.9130	0.9291	0.9314	0.9291	0.9222	
3	Logistic Regression	0.9153	0.8947	0.9291	0.9336	0.9336	0.9213	
0	SVM	0.9153	0.9016	0.9268	0.9314	0.9222	0.9195	
2	KNN	0.9085	0.9016	0.9291	0.9108	0.8879	0.9076	
4	Decision Tree	0.8970	0.8810	0.9176	0.9153	0.8810	0.8984	
1	Naive Bayes	0.7323	0.7368	0.7506	0.7254	0.7735	0.7437	

5.2 Final Train vs. Test Accuracy

Table 1: Final Train vs. Test Accuracy for All Models

Model	Train Accuracy	Test Accuracy
XGBoost	0.9931	0.9405
Gradient Boosting	0.9771	0.9314
Random Forest	0.9760	0.9245
Stacked Classifier	0.9811	0.9153
AdaBoost	0.9394	0.9153
Logistic Regression	0.9308	0.9153
K-Nearest Neighbors (KNN)	1.0000	0.9108
Support Vector Machine (SVM)	0.9331	0.9108
Decision Tree	0.9725	0.8902
Naive Bayes	0.7586	0.7254

6 Conclusion

- **Performance Hierarchy:** A clear performance hierarchy was observed. Simple probabilistic models like Naive Bayes performed the poorest, while complex ensemble methods performed the best. The high training accuracy on tree-based models suggests some overfitting, which could be addressed with further hyperparameter tuning.
- **Superiority of Ensemble Methods:** Tree-based ensemble models significantly outperformed all other individual classifiers, highlighting their ability to capture complex patterns in the data.
- **XGBoost as the Champion Model:** The **XGBoost Classifier** was unequivocally the best-performing model, achieving the highest test accuracy of **94.00%**. This demonstrates its robustness and state-of-the-art performance for this classification task.

7 Learning Outcomes

- Gained hands-on experience performing hyperparameter tuning for multiple classifiers including SVM, KNN, Naive Bayes, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, and XGBoost.
- Learned to apply PCA for dimensionality reduction and observed its impact on model performance, variance, and overfitting.
- Understood how different models respond to PCA, identifying which models benefit most (Naive Bayes, KNN) and which are robust to high-dimensional data (ensemble methods, linear models).
- Observed how stacking ensembles can improve predictive performance and maintain robustness to dimensionality reduction compared to individual models.

GitHub Repository

The complete source code is available on GitHub: [GitHub](#)